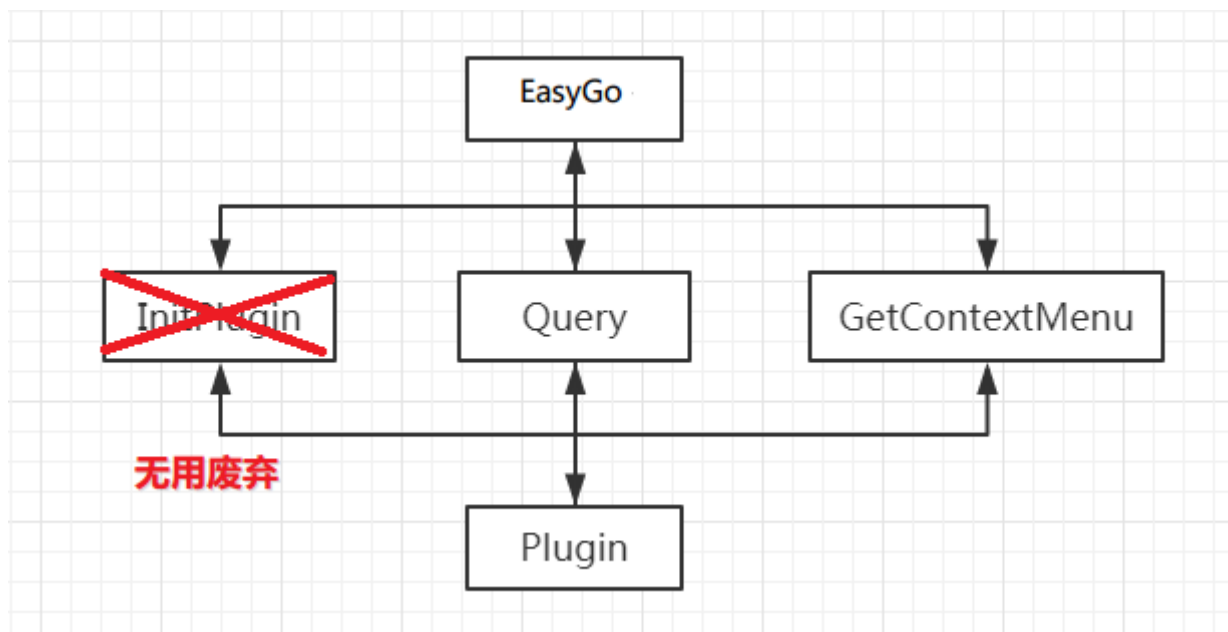


简介

EasyGo借鉴了wox的插件架构，尤其是python插件，几乎差不多，所以如果你开发过wox的python插件，那么可以很容易的将它移植到EasyGo上面来。一个python插件是由三部分组成，py源文件，plugin.json配置文件和应用图片。

架构

插件的整体架构如图，python插件由于InitPlugin没有实际作用，废弃



插件接口介绍

python的插件接口相对于c++要简单许多，因为针对python插件开发封装了一个python插件类供python插件继承使用

下面是一个简单的python插件示例，插件示例需要是utf8格式

```
1 import uuid
2 import clipboard
3 # PythonPlugin是python插件的封装类，文件位于JsonRPC下面
4 from PythonPlugin import PythonPlugin
5
6 class Main(PythonPlugin):
7
8     # 查询函数，使用EasyGo传入的查询关键字来获取结果列表并返回，返回值是数组
9     # [in] param: 用户输入查询参数，字符串类型
10    def Query(self, param):
11        # results为结果数组
```

```

12     results = []
13     results.append({
14         "Title": "Demo",                #这是EasyGo上行文字
15         "ShowType": 0,                  #这个是显示方式，默认
16         "SubTitle": "this is a Demo",    #这是EasyGo的下行文字
17         "IconPath": "img/app.png",        #这是结果图标，可以为空，则
18         "Action": {                      # 用户选中某
19             'FuncName': 'copy_to_clip',    # 函数名，如果是自定义函数
20             'Parameter': param,            # 函数参数， 函数只
21             'HideWindow': True             # 在响应用户操作后是否隐藏窗口
22         }
23     })
24
25     return results
26
27     @staticmethod
28     def copy_to_clip(self, text):
29         clipboard.copy(text)
30
31 if __name__ == '__main__':
32     Main()

```

EasyGo提供给插件的接口

EasyGo 为插件提供了一些现有的回调接口，它们的调用方法和上述的 `Action` 结构一样，将下面的函数名传入 `FuncName`，然后传入 `Parameter` 参数即可使用

`Ra_CopyPath`：会拷贝Parameter文字内容到剪切板

`Ra_Copy`：这个会拷贝Parameter指定的文件到剪切板，你可以在桌面上进行粘贴文件操作

`Ra_CopyImage`：拷贝图片到剪切板，可以进行图片粘贴操作

`Ra_Delete`：删除Paramter参数指定的文件或者文件夹（不可恢复，非移动到回收站，请注意）

`Ra_Recycle`：删除Paramter参数指定的文件或者文件夹到回收站

`Ra_Open`：打开Parameter指定的可执行程序或者文件夹

`Ra_OpenFileFolder`：打开Parameter指定文件路径所在的文件夹

`Ra_OpenWeb`：打开Parameter指定的网址

`Ra_ChangeQuery` : 会设置EasyGo输入框的值为Parameter字符串, 如果和之前输入框的文字不同会触发一次查询操作

`Ra_ShowMsg` : 会弹出消息提示框, 非阻塞

`Ra_ShowTip` : 会在系统右下角弹出提示框

`Ra_ShowContent` : 会弹出一个文本浏览框显示你传入的Parameter内容, 支持图文, 图片需要转为特殊的图片代码, 调用函数ToImageCode

`Ra_EditFile` : 会弹出一个文本框, 显示你传入的文件的内容, 纯文本, 支持编辑

`Ra_PlayMusic` : 播放本地或者网络音乐, 传入音乐地址, 再次调用则会暂停音乐

`Ra_PauseMusic` : 暂停当前播放的音乐

`Ra_StopMusic` : 停止当前播放的音乐

还有一种接口主要提供给python插件在自定义回调接口中使用, 不能使用在InitPlugin, Query和GetContextMenu里面, 使用这些接口需要

```
from PythonPlugin import PythonPluginAPI
```

目前提供的接口如下

`PythonPluginAPI.ChangeQuery(query)` : 修改输入框的查询字符串, query为字符串类型, 如果文本发生改变会触发重新查询

`PythonPluginAPI.Reload()` : 重载EasyGo

`PythonPluginAPI.ReQuery()` : 强制重新查询, 而不管输入框的文本是否发生变化

`PythonPluginAPI.ShowMsg(title, msg)` : 让EasyGo弹出消息框, title是标题, msg是消息内容

`PythonPluginAPI.ShowTip(title, msg)` : 让EasyGo托盘弹出消息框, title是标题, msg是消息内容

`PythonPluginAPI.ShowContent(title, msg)` : 让EasyGo弹出文本阅读框, msg为文本内容

`PythonPluginAPI.EditFile(title, filepath)` : 让EasyGo弹出文本编辑框, 编辑文件内容, filepath是文件完整路径

`PythonPluginAPI.ToImageCode` : 将本地或者网络路径转换为图片代码并返回代码文本, 可以查看JsonRPC下的pythonsdk代码实现

`PythonPluginAPI.Ra_PlayMusic(musicPath)` : 播放本地或者网络音乐, 传入音乐地址, 再次调用则会暂停音乐

`PythonPluginAPI.Ra_PauseMusic()` : 暂停当前播放的音乐

PythonPluginAPI.Ra_StopMusic(): 停止当前播放的音乐

配置文件

```
1 {
2     "ID": "98451A7B5C764F6A9A3814EF7E4AFEEC",          这个是guid唯一标示
3     "Keyword": "guid",                                  这个是插件关键字，用户使用该默认关键字出发插件
4     "Argc": 1,                                           这个是代表插件参数个数，关键字+子关键字
5     "Name": "生成Guid",                                  插件名
6     "Description": "生成Guid",                          插件描述
7     "Author": "zsxxxx",                                  插件作者
8     "Version": "1.0.0",                                  插件版本
9     "PluginType": "python",                              插件类型，目前有c++，python和e三种
10    "PluginMode": "EnterMode",                          插件模式，EnterMode或者RealMode，此字段可以自定义
11    "AcceptType": "png,ico,folder",                      插件支持的拖入的文件类型，就是文件的后缀名，
12    "EnableSeparate": 1,                                  是否开启插件分离窗口的支持，默认为1开启
13    "IconPath": "img\\app.png",                          插件图标
14    "ExeName": "Main.py",                                插件源文件名字
15    "CfgPath": "set.json"                                插件的配置选项，可以用相对路径，可以指定配置文件名也可
16 }
```

插件CfgPath配置文件

配置文件json，配置项均为文本格式，而且不支持多级，类似map结构

```
1 {
2     "name": "admin",
3     "password": "admin",
4     "role": "admin"
5 }
```

插件打包

插件打包后的格式是 **插件名.Plugin**，其实就是将插件的三个文件打包成zip文件，然后把zip后缀改成Plugin即可，记住了，打包的时候插件的外层不要有文件夹

然后如果你的插件需要依赖一些Python库，那么请放入 **requirements.txt**，一个完成的包如图：



其中requirements.txt里面，每个包名占一行，包不建议加上版本号，另外如果你不依赖其他库不要加入 requirements.txt 不然影响安装速度

如果你有说明文件，那么请加入 `readme.txt` 文件，全小写，然后加入帮助内容，用户在插件管理页面可以通过右键详情打开帮助文档，然后你也可以放入手动配置文件，在 plugin.json指定（见上），然后用户可以在插件管理页面通过右键设置打开配置文件进行配置

具体的使用方法请参考示例